



InsightPulse

TECHNICAL ARCHITECTURE & SOLUTION DESIGN

InsightPulse Platform Technical Design Document

AI-Powered Multi-Channel Customer & Employee Experience Survey
Platform

DOCUMENT TYPE

Technical Architecture Document (TAD)

VERSION

1.0

DATE OF ISSUE

16 June 2026

CLASSIFICATION

Confidential — Internal

PLATFORM

rest-express · skill-pulse.io

STATUS

Issued for Review

InsightPulse Engineering · Solution Architecture Practice

© 2026 — All rights reserved

Document Control

Distribution & Classification

This document provides a comprehensive technical description of the InsightPulse platform — its architecture, data model, integration surfaces, security posture, and operational characteristics. It is intended for solution architects, engineering leads, platform operators, and technical stakeholders evaluating or extending the system.

Version History

VERSION	DATE	AUTHOR	DESCRIPTION OF CHANGE
0.1	10 Jun 2026	Engineering	Initial draft — architecture and data model
0.9	14 Jun 2026	Solution Architecture	Added integration, security & analytics sections
1.0	16 Jun 2026	Solution Architecture	Issued for review — full document baseline

Reviewers & Approvers

ROLE	RESPONSIBILITY	STATUS
Solution Architect	Architecture integrity, design decisions	OWNER
Engineering Lead	Implementation feasibility, code alignment	PENDING
Security Officer	RBAC, data protection, compliance review	PENDING
Product Owner	Business capability sign-off	PENDING

Confidentiality Notice. This document contains proprietary architecture and design information relating to the InsightPulse platform. It must not be distributed outside the authorised recipient list without the written consent of the document owner.

Table of Contents

1	Executive Summary	03
2	Solution Overview & Business Context	04
	2.1 Platform Capabilities · 2.2 Target Users · 2.3 Channel Strategy	04
3	System Architecture	05
	3.1 Architectural Style · 3.2 Logical View · 3.3 Technology Stack	05
4	Application Design	07
	4.1 Frontend · 4.2 Backend & Routing · 4.3 Services Layer · 4.4 Storage Abstraction	07
5	Data Architecture	09
	5.1 Domain Model · 5.2 Core Tables · 5.3 Analytics Pipeline	09
6	Survey Response & Analytics Pipeline	11
7	Integration Architecture	12
	7.1 VAPI Voice · 7.2 WhatsApp · 7.3 Zoho CRM · 7.4 OpenRouter · 7.5 Email	12
8	Security & Access Control	14
9	Quality, Testing & Operations	15
10	Risks, Constraints & Recommendations	16
A	Appendix — API Surface & Glossary	17

1 Executive Summary

InsightPulse is an AI-powered, multi-channel survey and experience-management platform that captures customer and employee feedback across **four distinct channels** — Voice (via VAPI), WhatsApp (via Meta Cloud API), Web/Link, and Email — and converts that feedback into real-time, AI-enriched analytics. The platform is delivered as a single, cohesive full-stack TypeScript application combining a React front end, an Express API back end, and a shared, strongly-typed database schema.

The defining characteristic of the platform is its **unified analytics pipeline**: regardless of the channel through which a response arrives, every submission converges on a single pair of database tables and is enriched by a Large Language Model (LLM) to produce sentiment, emotion, theme, and recommendation data. This guarantees a single source of truth for all dashboards and eliminates channel-specific reporting silos.

4
FEEDBACK CHANNELS

65
DATABASE TABLES

257
API ENDPOINTS

33
ROUTE MODULES

22
DOMAIN SERVICES

4
EXPERIENCE METRICS (CSAT/NPS/CES/EVI)

1.1 Key Architectural Decisions

DECISION	RATIONALE
Single-package monorepo (client + server + shared)	Shared Drizzle/Zod types across the wire; one build, one deploy unit; eliminates type drift between front and back end.
Unified response model (<code>responses</code> + <code>feedback_analytics</code>)	All channels write the same two tables, so analytics never depend on channel of origin — one source of truth.
Thin routes, rich services	Domain logic isolated in <code>server/services/*</code> ; routes orchestrate only — improves testability and reuse.
Storage abstraction singleton	<code>storageFactory.ts</code> centralises persistence behind one interface (Drizzle/Postgres), decoupling routes from the ORM.
LLM with primary + fallback model	OpenRouter routes to deepseek-chat with gemini-2.0-flash fallback for resilience; responses persist even if enrichment fails.
Fire-and-forget department aggregation	Department NPS recomputes asynchronously on each submission without blocking the response path.

1.2 Document Purpose

This Technical Architecture Document establishes the authoritative reference for the platform's structure and behaviour. It supports onboarding, change governance, security review, and capacity planning. Sections 2–7 describe the system as built; Sections 8–10 address cross-cutting concerns and forward-looking recommendations.

2 Solution Overview & Business Context

InsightPulse addresses a core enterprise challenge: feedback arrives through fragmented channels and rarely produces comparable, actionable intelligence. The platform consolidates collection and applies a consistent AI analysis layer so that leadership receives one coherent view of customer and employee experience.

2.1 Platform Capabilities

CAPABILITY AREA	DESCRIPTION
Multi-Channel Distribution	Voice (AI phone surveys), WhatsApp (sequential question delivery), Web/Link forms, and Email-distributed survey links.
Real-Time Analytics	Live dashboard auto-refreshed every 30 seconds across 5 analytics tabs (Overview, CSAT/NPS, Customer Journey, Trends, Insights).
Experience Metrics	CSAT, NPS (with promoter/passive/detractor split), CES (1–5 normalised to 0–100%), and a proprietary EVI (Emotional Value Index, 0–100).
AI Sentiment & Emotion	8-emotion Plutchik model, theme extraction, urgency scoring, insights and recommendations — all LLM-generated.
CRM Integration	Zoho CRM OAuth for contact sync and targeted survey distribution.
RBAC & Governance	Granular roles/permissions, organization → department hierarchy, audit logs, session management, SSO provider support.
Retention & Risk	Employee turnover-risk detection, action planning boards, leadership insights, and executive summaries.
Internationalisation	Bilingual English / Arabic UI (i18next), including right-to-left support.

2.2 Target Users

Experience & CX Leaders Consume dashboards, trends, and AI recommendations to drive experience strategy.	HR / People Teams Monitor employee sentiment, turnover risk, and department-level NPS; run action plans.
Survey Administrators Build surveys, schedule campaigns, manage templates and multi-channel distribution.	Platform Administrators Manage RBAC, organizations, integrations, audit and compliance.

2.3 Channel Strategy

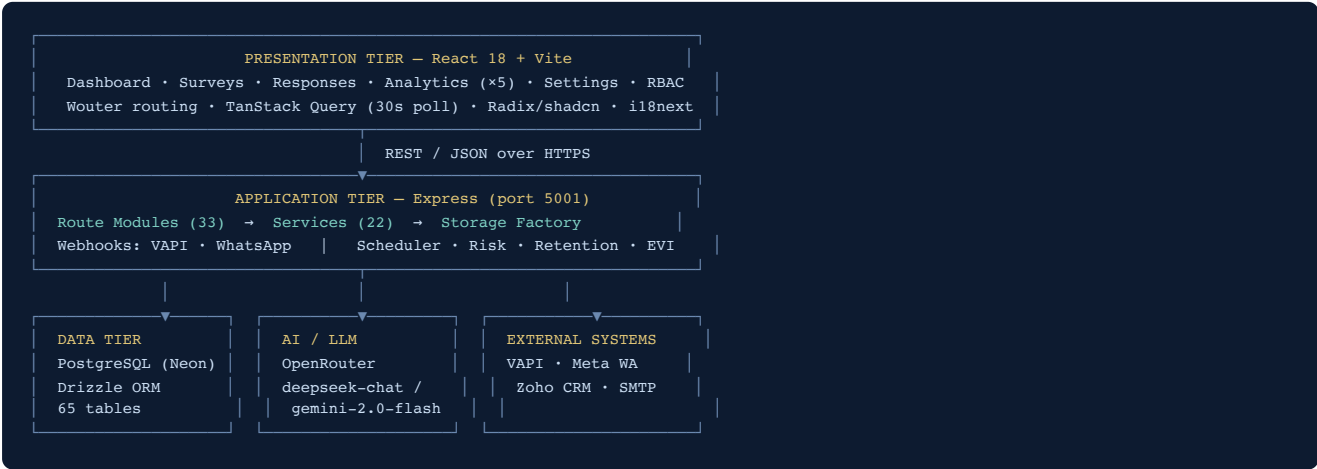
Each channel is purpose-built around its medium but converges on a shared response contract. Voice surveys leverage VAPI structured outputs to return all four scores directly without a secondary LLM pass; WhatsApp delivers questions sequentially through the Meta Cloud API; Web/Link and Email rely on interactive forms with real-time validation. This "diverge at the edge, converge at the core" pattern is the platform's central design motif.

3 System Architecture

3.1 Architectural Style

InsightPulse follows a **modular monolith** architecture packaged as a single deployable unit (npm package `rest-express`). The codebase is partitioned into three logical zones — `client/`, `server/`, and `shared/` — that share a common type system but are bundled separately at build time (Vite for the client, esbuild for the server). This approach delivers the developer velocity and type-safety of a monorepo while avoiding the operational overhead of distributed microservices.

3.2 Logical View



3.3 Technology Stack

LAYER	TECHNOLOGY	VERSION	ROLE
Frontend	React	18.3	UI rendering
	Vite	5.4	Build & dev server
	Wouter	3.3	Client-side routing
	TanStack Query	5.60	Server state / polling
	Radix UI + shadcn + Tailwind	3.4	Component system & styling
	i18next	25.8	EN/AR internationalisation
Backend	Node.js	22.x	Runtime (18+ supported)
	Express	4.21	HTTP API framework
	TypeScript	5.6	Language (ESM throughout)
	Passport + express-session	0.7	Authentication & sessions
Data	PostgreSQL (Neon serverless)	15+	Primary datastore
	Drizzle ORM	0.39	Type-safe queries & schema
	Zod + drizzle-zod	3.24	Runtime validation
AI / Charts	OpenRouter SDK	0.5	LLM gateway
	Recharts / Chart.js / D3	—	Data visualisation
Testing	Vitest	4.0	Unit & integration tests

Build & Deployment. `npm run build` performs a two-stage build: Vite compiles the client to `dist/public`, then esbuild bundles the server to `dist/index.js` as an ESM module with external packages. In development, both tiers share port 5001 via Vite middleware mounted on Express. A `Dockerfile` and `cloudbuild.yaml` support containerised deployment to Google Cloud.

Application Design

4.1 Frontend Architecture

The application maintains **two UI generations**. The current, Figma-derived UI lives under `client/ui/` (exposed via the `@ui` alias) and renders all root paths (`/dashboard` , `/surveys` , `/analytics` , `/settings`). A legacy UI under `client/src/pages/` is retained at `/ui/*` paths for reference only. All new development targets `client/ui/`.

The front end comprises **28 page components** and a reusable component library. Pages are routed from `client/src/App.tsx`. Server state is managed exclusively through TanStack Query; the dashboard's `useUnifiedMetrics` hook polls `/api/unified-analytics/metrics` every 30 seconds, keeping all analytics views live without manual refresh.

PAGE GROUP	REPRESENTATIVE COMPONENTS
Dashboard & Analytics	DashboardPage, AnalyticsOverviewPage, AnalyticsTrendsPage, CustomerJourneyPage, InsightsPage, ViewAnalyticsPage
Survey Lifecycle	SurveysPage, SurveyBuilderPage, SurveySharePage, SurveyResponsesPage, SurveysTemplatesPage, SurveyDesktopViewPage
Action & Insights	ActionPlanningPage, ActionPlanningBoardPage, TeamInsightsPage, ReportsPage
Settings & Identity	SettingsPage, SettingsSecurityPage, SettingsCrmIntegrationsPage, SettingsLanguagePage, ProfilePage, LoginPage

4.2 Backend & Routing

The Express application is bootstrapped in `server/index.ts`, which registers core routes via `registerRoutes(app)` and then mounts **33 dedicated route modules** from `server/routes/*`, each scoped to a feature area. New feature areas follow the convention of a dedicated `xxxRoutes.ts` file mounted explicitly. In total the API exposes approximately **257 endpoints**. The survey scheduler service is started at boot.

DOMAIN	ROUTE MODULES
Surveys & Responses	surveyResponseRoutes, surveyAnalyticsRoutes, surveyScheduleRoutes, voiceRoutes, whatsappRoutes, emailSurveyRoutes, anonymousFeedbackRoutes
Analytics	unifiedAnalyticsRoutes, analyticsRoutes, analyticsDataRoutes, advancedAnalyticsRoutes, analyticsHeatmapRoutes, turnoverRateAnalyticsRoutes
Identity & Access	rbacRoutes, adminRoutes, advancedPermissionsRoutes, sessionRoutes, passwordResetRoutes, superuserRoutes, onboardingRoutes
Org & Integration	organizationRoutes, departmentRoutes, crmIntegrationRoutes, dataTransferRoutes, auditAndImportRoutes, subscriptionRoutes
Insight & Action	actionPlanningRoutes, teamInsightsRoutes, eviRoutes, dashboardRoutes, assessmentPeriodRoutes, turnoverRiskRoutes

4.3 Services Layer

Business logic is concentrated in **22 domain services** under `server/services/*`. Routes remain thin and delegate to services; most services depend on the OpenRouter AI gateway and the Drizzle `db` handle.

SERVICE	RESPONSIBILITY
<code>unifiedAnalyticsService</code>	Aggregates all metrics from the DB — trend deltas, themes, 8-emotion daily trends, CES normalisation, insights bundle.
<code>departmentNpsService</code>	Resolves respondent → department and re-aggregates NPS per department/month.
<code>surveyResponseAnalysisService</code>	Runs LLM analysis on web/WhatsApp responses (sentiment, emotions, themes).
<code>structuredOutputService</code>	Parses VAPI structured outputs into the four experience scores.
<code>eviScoringService</code>	Computes the Emotional Value Index.
<code>surveySchedulerService</code>	Boot-time scheduler that dispatches scheduled survey campaigns.
<code>riskDetectionService</code> / <code>employeeTurnoverRiskService</code>	Detects attrition risk signals from feedback.
<code>actionPlanningService</code> / <code>leadershipInsightsProcessor</code>	Generates action plans and leadership-level insights from analytics.
<code>executiveSummaryService</code> / <code>dailyInsightsService</code>	Produces executive summaries and daily insight digests.
<code>systemHealthMonitor</code> / <code>monitoringService</code>	Operational health and metrics monitoring.

4.4 Storage Abstraction

All persistence flows through `server/storageFactory.ts`, a singleton that exports a `storage` object and always returns a `DatabaseStorage` implementation backed by Drizzle/Postgres (`server/db.ts`). This abstraction decouples the rest of the server from the ORM and provides a single seam

for persistence concerns. The legacy Caspio integration has been fully removed.

5 Data Architecture

The schema (`shared/schema.ts` , ~1,380 lines) defines **65 tables** across nine functional domains, managed through Drizzle's push workflow (`npm run db:push`) supplemented by hand-authored SQL migrations under `migrations/` .

5.1 Domain Model

DOMAIN	REPRESENTATIVE TABLES
Organization & People	<code>organizations</code> , <code>departments</code> , <code>employees</code> , <code>users</code>
Survey Core	<code>surveys</code> , <code>responses</code> , <code>notifications</code> , <code>survey_templates</code> , <code>survey_schedules</code>
Analytics	<code>feedback_analytics</code> , <code>structured_outputs</code> , <code>analytics_surveys</code> , <code>analytics_questions</code> , <code>analytics_responses</code> , <code>analytics_metrics</code> , <code>department_nps</code> , <code>trends_analysis</code> , <code>segmentation_analysis</code> , <code>benchmarks</code>
RBAC & Security	<code>roles</code> , <code>permissions</code> , <code>role_permissions</code> , <code>user_roles</code> , <code>department_roles</code> , <code>role_hierarchy</code> , <code>permission_groups</code> , <code>elevated_permissions</code> , <code>permission_violations</code>
Audit & Identity	<code>audit_logs</code> , <code>activity_logs</code> , <code>authentication_events</code> , <code>sessions</code> , <code>password_reset_tokens</code> , <code>sso_providers</code> , <code>user_invites</code> , <code>user_import_batches</code>
CRM	<code>crm_configs</code> , <code>crm_sync_logs</code> , <code>crm_tags</code> , <code>crm_contact_tags</code>
Insight & Action	<code>executive_summary</code> , <code>action_plans</code> , <code>action_items</code> , <code>action_planning</code> , <code>leadership_insights</code> , <code>interventions</code> , <code>recognition_templates</code> , <code>daily_insights</code>
EVI & Risk	<code>assessment_periods</code> , <code>evi_assessments</code> , <code>evi_results</code> , <code>risk_detection_batches</code> , <code>risk_detection_results</code>
Reporting & Ops	<code>reports</code> , <code>report_schedules</code> , <code>report_archives</code> , <code>export_jobs</code> , <code>data_transfer_jobs</code> , <code>monitoring_metrics</code> , <code>onboarding_progress</code>

5.2 Core Response Tables

The platform's analytical integrity rests on two convergent tables written by every channel:

TABLE	PURPOSE	KEY COLUMNS
<code>responses</code>	Raw answers + scalar scores	<code>eviScore</code> , <code>npsScore</code> , <code>csatScore</code> , <code>cesScore</code> , <code>respondentPhone</code> , <code>overallSentiment</code> , <code>analysisSummary</code>
<code>feedback_analytics</code>	Full LLM enrichment payload	<code>emotions</code> (JSONB, 8 Plutchik emotions), <code>insights[]</code> , <code>recommendations[]</code> , <code>category</code> , <code>urgency</code> , <code>aiAnalysis</code> , <code>source</code>
<code>structured_outputs</code>	VAPI raw scores	<code>npsScore</code> , <code>cesScore</code> , <code>eviScore</code> (integers)
<code>department_nps</code>	Auto-aggregated department NPS	<code>promoters</code> / <code>passives</code> / <code>detractors</code> counts per department / month

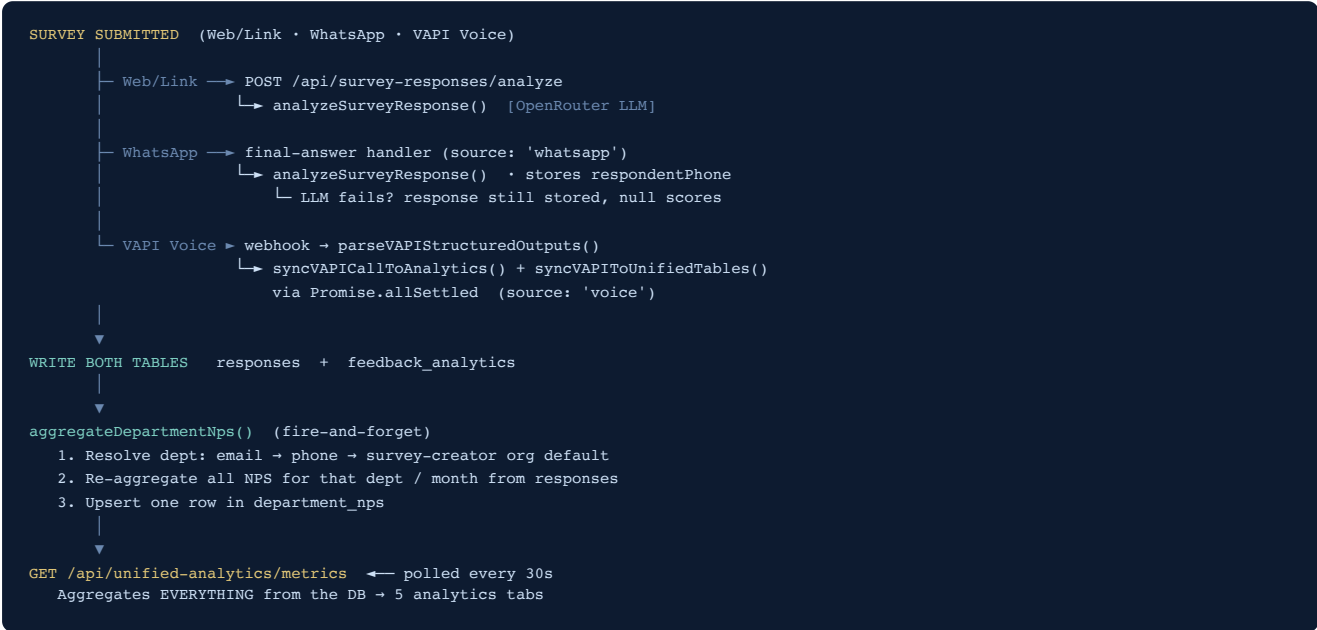
Nullable enrichment. Score columns in `feedback_analytics` (`eviScore` , `npsScore` , `csatScore` , `emotions` , `insights` , `recommendations`) are nullable by design — the VAPI and WhatsApp paths may carry partial data, and a response is never discarded merely because enrichment was incomplete.

5.3 Schema Governance

Schema evolution follows the Drizzle *push* model rather than generated migration files: developers edit `shared/schema.ts` and run `npm run db:push` . Ad-hoc structural changes and data backfills are captured as raw SQL under `migrations/` (organised into `core/` , `features/` , and `meta/`), executed via dedicated runner scripts.

6 Survey Response & Analytics Pipeline

The response-to-analytics pipeline is the operational heart of InsightPulse. Every channel writes the same two tables and then triggers an asynchronous department-NPS re-aggregation.



6.1 Channel-Specific Handling

CHANNEL	SCORE SOURCE	NOTES
Web / Link	LLM (OpenRouter)	<code>analyzeSurveyResponse()</code> produces all enrichment; canonical path.
WhatsApp	LLM (OpenRouter)	Same analysis path; stores <code>respondentPhone</code> ; resilient to LLM failure (null scores).
VAPI Voice	VAPI structured outputs	All four scores returned directly — no extra LLM pass; dual-write via <code>Promise.allSettled</code> .

6.2 Analytics Consumption

All five analytics tabs read **exclusively** from `GET /api/unified-analytics/metrics`. The endpoint computes period-over-period trend deltas, theme tables (from `feedback_analytics.category` + sentiment), 8-emotion daily trends (from `emotions` JSONB), the CES score (1–5 normalised to 0–100%), and a top-insights/recommendations bundle.

TAB	KEY DATA SHOWN
Overview	EVI, CSAT, NPS, response rate, sentiment breakdown, themes table, top insights
CSAT / NPS	CSAT score + delta, CES gauge, promoters %, NPS donut
Customer Journey	Category themes — mentions, sentiment, example per theme
Trends	EVI/NPS/CSAT scores + deltas, satisfaction distribution, engagement sub-cards
Insights	NPS donut, CSAT area chart, EVI line chart, response volume, recommended actions

Design constraint. No hardcoded or random values are permitted in the Analytics UI. Every figure must derive from the unified metrics endpoint, which aggregates exclusively from the database. This guarantees that displayed analytics are always traceable to real response data.

7 Integration Architecture

InsightPulse integrates with five external systems. Webhook-based integrations rely on the `PUBLIC_BASE_URL` environment variable for callback routing.

7.1 VAPI — Voice Surveys

Handled by `voiceRoutes.ts`. All four scores (CSAT, NPS, EVI, CES) are returned via VAPI *structured outputs* and stored directly, eliminating a secondary LLM pass. Parsing occurs in `structuredOutputService.parseVAPIStructuredOutputs()`; the webhook runs the legacy sync and the unified-tables sync concurrently through `Promise.allSettled`.

7.2 Meta WhatsApp Cloud API

Handled by `whatsappRoutes.ts` (the previous WATI integration is fully removed). Questions are delivered sequentially; the final-answer handler invokes the standard analysis path with `source: 'whatsapp'`.

ENVIRONMENT VARIABLE	PURPOSE
<code>META_WA_ACCESS_TOKEN</code>	Graph API access token
<code>META_WA_PHONE_NUMBER_ID</code>	Sending phone number identifier
<code>META_WA_VERIFY_TOKEN</code>	Webhook verification token
<code>META_WA_GRAPH_VERSION</code>	Graph API version (default <code>v21.0</code>)

7.3 Zoho CRM — OAuth

Handled by `crmIntegrationRoutes.ts`. Uses OAuth to fetch and sync contacts into `crm_configs` / `crm_sync_logs`, enabling targeted survey distribution to CRM contacts. Tagging is supported via `crm_tags` and `crm_contact_tags`.

7.4 OpenRouter — AI / LLM Gateway

The single gateway for all AI enrichment. Primary model is **deepseek-chat** with **gemini-2.0-flash** as fallback, used for web/WhatsApp response analysis and retention insights. The dual-model strategy provides resilience against provider-side failures.

7.5 Email & SMTP

Handled by `emailSurveyRoutes.ts` using `nodemailer` for survey-link distribution; a Gmail integration is documented under `docs/GMAIL_INTEGRATION.md`.

7.6 Integration Summary

SYSTEM	DIRECTION	PROTOCOL	RESILIENCE
VAPI	Inbound webhook	HTTPS / structured output	CONCURRENT DUAL-WRITE
Meta WhatsApp	Bi-directional	Graph API + webhook	STORE-ON-LLM-FAILURE
Zoho CRM	Outbound + sync	OAuth 2.0 REST	SYNC LOGS
OpenRouter	Outbound	HTTPS	PRIMARY + FALLBACK MODEL
SMTP / Gmail	Outbound	SMTP	STANDARD RETRY

8 Security & Access Control

8.1 Authentication

Authentication uses Passport with a local strategy over `express-session` ; passwords are hashed with `bcrypt` . Sessions are persisted (via `connect-pg-simple` / memory store) and recorded in the `sessions` table. JWT support (`jsonwebtoken`) and OpenID Connect (`openid-client`) are present for token and SSO flows, with `sso_providers` modelling federated identity. Password resets are token-based via `password_reset_tokens` and `passwordResetRoutes.ts` .

8.2 Role-Based Access Control

RBAC is comprehensive and hierarchical. The model spans roles, permissions, role–permission mappings, and user–role assignments, with department-scoped roles and a role hierarchy for inheritance. Temporary privilege escalation is modelled explicitly.

CONCERN	TABLES
Core RBAC	<code>roles</code> , <code>permissions</code> , <code>role_permissions</code> , <code>user_roles</code>
Hierarchy & Scope	<code>department_roles</code> , <code>role_hierarchy</code> , <code>permission_groups</code>
Elevation & Violations	<code>elevated_permissions</code> , <code>permission_violations</code>

RBAC is administered through `rbacRoutes.ts` and `advancedPermissionsRoutes.ts` ; a dedicated test suite (`tests/rbac.test.ts`) validates enforcement.

8.3 Audit & Traceability

The platform maintains layered audit trails: `audit_logs` for administrative actions, `activity_logs` for user activity, and `authentication_events` for sign-in telemetry. Bulk user onboarding is tracked through `user_import_batches` and `user_invites` , providing a complete record of identity lifecycle events.

8.4 Data Protection & Secrets

CONTROL	IMPLEMENTATION
Secrets management	Environment variables (<code>.env</code> , git-ignored); integration tokens for VAPI, Meta, Zoho, OpenRouter held outside source.
Transport security	HTTPS across all external integrations and webhooks (<code>PUBLIC_BASE_URL</code>).
Credential storage	<code>bcrypt</code> password hashing; no plaintext credentials persisted.
Input validation	Zod schemas (<code>shared/schemas.ts</code>) validate request payloads at the boundary.
Anonymity option	<code>anonymous_feedback</code> table + dedicated route supports unattributed submissions.

Recommendation. Formalise secret rotation and consider a managed secrets store (e.g. Google Secret Manager) for production, given the breadth of third-party tokens. See Section 10.

9 Quality, Testing & Operations

9.1 Testing Strategy

Testing uses Vitest with a dedicated configuration (`tests/vitest.config.ts`). The suite separates business-logic unit tests from integration tests spanning authentication, dashboard, surveys, analytics, admin, RBAC, and CRM/feedback flows.

COMMAND	SCOPE
<code>npm test</code>	Full suite (all tests)
<code>npm run test:unit</code>	Business-logic unit tests
<code>npm run test:integration</code>	Auth, dashboard, surveys, analytics, admin, RBAC, CRM/feedback
<code>npm run test:coverage</code>	Coverage report
<code>npm run check</code>	TypeScript typecheck (no emit)

9.2 Operational Tooling

CONCERN	MECHANISM
Health monitoring	<code>systemHealthMonitor</code> + <code>monitoringService</code> ; <code>monitoring_metrics</code> table
Scheduled work	<code>surveySchedulerService</code> started at boot; <code>survey_schedules</code> table
Data seeding	<code>seed:analytics</code> , <code>seed:employees</code> ; organised under <code>scripts/setup/</code>
Data transfer / export	<code>data_transfer_jobs</code> , <code>export_jobs</code> ; <code>dataTransferRoutes.ts</code>
Reporting	<code>reports</code> , <code>report_schedules</code> , <code>report_archives</code> ; client-side PDF via jsPDF

9.3 Build & Deployment Operations

STAGE	COMMAND / ARTEFACT
Development	<code>npm run dev</code> — tsx + Vite middleware on port 5001
Build	<code>npm run build</code> — Vite (client → <code>dist/public</code>) + esbuild (server → <code>dist/index.js</code>)
Production run	<code>npm start</code> — <code>NODE_ENV=production node dist/index.js</code>
Schema sync	<code>npm run db:push</code> (Drizzle push)
Container / Cloud	<code>Dockerfile</code> + <code>cloudbuild.yaml</code> (Google Cloud Build, <code>gcp-build</code>)

9.4 Code Conventions

- ESM throughout (`"type": "module"`); relative cross-boundary imports use `.ts` extensions.
- New endpoints follow the dedicated `xxxRoutes.ts` + explicit-mount convention.
- Database access flows through `storage` or `db` + `@shared/schema` .
- Path aliases: `@/` → `client/src`, `@shared/` → `shared`, `@ui/` → `client/ui`, `@assets/` → `attached_assets`.
- Scripts organised by purpose: `scripts/database/` , `scripts/setup/` , `scripts/utils/` , `scripts/test-data/` .

10 Risks, Constraints & Recommendations

10.1 Architectural Constraints

CONSTRAINT	IMPLICATION
Modular monolith	Single deploy unit simplifies operations but couples scaling of front and back end; horizontal scaling is all-or-nothing.
Drizzle push (no migration files)	Fast iteration, but weaker change auditability than versioned migrations; mitigated by SQL files in <code>migrations/</code> .
LLM dependency	Enrichment quality depends on OpenRouter availability; mitigated by primary+fallback models and store-on-failure.
Dual UI generations	Legacy UI under <code>/ui/*</code> increases surface area; should be retired once parity is confirmed.

10.2 Risk Register

RISK	LIKELIHOOD	IMPACT	MITIGATION
External provider outage (VAPI/Meta/OpenRouter)	MED	HIGH	Fallback model; responses persist with null scores; retry queues.
Secret leakage via env mishandling	LOW	HIGH	Managed secret store; rotation policy; least-privilege tokens.
Schema drift across environments	MED	MED	Versioned migrations; CI schema validation.
Analytics polling load at scale	MED	MED	Caching/materialised metrics; push (WebSocket/SSE) over 30s poll.
LLM cost growth with volume	MED	MED	Batch enrichment; cache repeat analyses; monitor token spend.

10.3 Forward Recommendations

1. **Adopt versioned migrations** for production environments to strengthen change governance and rollback.
2. **Introduce a metrics cache / materialised view** for `/api/unified-analytics/metrics` to decouple dashboard load from raw aggregation cost.
3. **Move from 30-second polling to server push** (SSE or WebSocket) for true real-time dashboards at lower request volume.
4. **Centralise secrets** in a managed store with rotation, given the number of third-party integrations.
5. **Retire the legacy UI** once the Figma-based UI reaches full functional parity, reducing maintenance surface.
6. **Add observability** (structured logging, tracing, dashboards) around the LLM and webhook paths, which are the highest-variance components.

10.4 Conclusion

InsightPulse is a well-structured, cohesively-typed platform whose central design strength — the convergence of all channels onto a single, AI-enriched response model — directly serves its business goal of unified experience intelligence. The recommendations above are evolutionary, not corrective: they harden an already sound architecture for scale, governance, and operational maturity.

A Appendix — API Surface & Glossary

A.1 API Surface (Selected Mount Points)

BASE PATH	MODULE	RESPONSIBILITY
/api/unified-analytics	unifiedAnalyticsRoutes	Single metrics endpoint feeding all dashboards
/api/survey-responses	surveyResponseRoutes	Web/link response ingestion + LLM analysis
/api (webhook)	whatsappRoutes	WhatsApp inbound message + final-answer handling
/api (webhook)	voiceRoutes	VAPI voice survey webhook
/api/rbac	rbacRoutes	Roles, permissions, assignments
/api/admin	adminRoutes, auditAndImportRoutes	Administration, audit, bulk import
/api/superuser	superuserRoutes	Platform-level superuser operations
/api/departments	departmentRoutes	Department management
/api/analytics	analyticsDataRoutes, surveyAnalyticsRoutes	Analytics data & per-survey analytics
/api/dashboard	dashboardRoutes	Dashboard aggregates
/api/team-insights	teamInsightsRoutes	Team-level insight generation
/api/evi	eviRoutes	Emotional Value Index assessments
/api/auth	passwordResetRoutes	Password reset flows
/api (CRM)	crmIntegrationRoutes	Zoho CRM OAuth + sync

The platform exposes approximately 257 endpoints across 33 route modules; the table above lists principal mount points registered in `server/index.ts`.

A.2 Glossary

TERM	DEFINITION
CSAT	Customer Satisfaction score, tracked with period-over-period delta.
NPS	Net Promoter Score, with promoter / passive / detractor breakdown.
CES	Customer Effort Score, captured 1–5 and normalised to 0–100%.
EVI	Emotional Value Index (0–100), a proprietary emotion-derived metric.
Plutchik emotions	The 8-emotion model stored as JSONB in <code>feedback_analytics.emotions</code> .
VAPI	Voice AI provider returning structured experience scores for phone surveys.
OpenRouter	LLM gateway routing to deepseek-chat (primary) / gemini-2.0-flash (fallback).
Unified analytics	The single metrics endpoint aggregating all channels from the database.
RBAC	Role-Based Access Control — hierarchical permission model.
Drizzle push	Schema-sync workflow applying <code>shared/schema.ts</code> directly to the database.

End of Document. InsightPulse Technical Architecture Document, Version 1.0 — issued 16 June 2026. Classified Confidential — Internal. For questions or change requests, contact the Solution Architecture practice.